

언어 모델은 어떻게 답을 만들까

Oct 8, 2026 · @.

들어가며: 손글씨 인식과 뼈대가 같다

ChatGPT나 Claude 같은 언어 모델은 3.6절 손글씨 인식과 **같은 일**을 해요. 숫자로 바꾸고, 곱하고 더하고, 소프트맥스로 확률을 내서, 가장 그럴듯한 것 하나를 고르는 거예요.

다른 점은 딱 두 가지예요.

- **고르는 대상이 달라요.** MNIST는 숫자 0~9 중 하나를 골랐어요. 언어 모델은 **단어 조각 수만 개** 중 하나를 골라요.
- **한 번에 끝나지 않아요.** MNIST는 이미지 한 장에 답 하나로 끝이에요. 언어 모델은 단어 조각 하나를 고르고, 그걸 붙여서 또 고르고, 이걸 **수백 번 반복**해서 답변 한 문단을 만들어요.

한 문장으로 말하면, 언어 모델은 **"다음에 올 단어 조각 맞추기"**를 엄청 잘하는 신경망이에요. 질문에 답하는 것도, 글을 쓰는 것도, 전부 이 한 가지 동작의 반복이에요.

단계	MNIST (3.6절)	언어 모델
입력	손글씨 이미지 한 장	지금까지의 대화 글
숫자로 바꾸기	픽셀 밝기 784개	토큰 번호 → 토큰마다 숫자 수천 개 (1, 2단계)
신경망 계산	은닉층 2개 (50개, 100개)	트랜스포머 층 수십 개 (3단계)
출력층 칸 수	10칸 (숫자 0~9)	수만 칸 (토큰 사전 크기) (4단계)
소프트맥스	10개 확률	수만 개 확률
고르기	argmax	argmax 또는 추첨 (5단계)
반복	없음, 한 번에 끝	고른 토큰을 붙여서 계속 (6단계)

아래는 이 표의 오른쪽 열을 한 단계씩 따라가요. 예시 질문은 끝까지 하나로 통일할게요.

예시 질문: "소프트맥스가 뭐야?"

1단계 토큰화: 글자를 번호로

신경망은 숫자만 계산할 수 있어요. MNIST에서 그림을 픽셀 밝기(0~255)로 바꿨듯이, 글도 숫자로 바뀌어야 해요. 그 첫걸음이 **토큰화**예요.

문장을 조각(토큰)으로 자른다

모델마다 미리 만들어둔 **토큰 사전**이 있어요. 문장을 이 사전에 있는 조각들로 잘라요.

"소프트맥스가 뭐야?"

↓ 조각내기

["소프트", "맥스", "가", " ", "뭐", "야", "?"]

조각은 단어 하나일 수도, 단어의 일부일 수도 있어요. 자주 쓰는 말은 통째로 한 조각, 드문 말은 여러 조각으로 나뉘요. 위의 자르는 위치는 설명을 위한 예시이고, 실제로는 모델마다 다르게 잘라요.

조각마다 번호를 붙인다

사전에는 조각마다 고유 번호가 있어요. 출석부 번호처럼요.

["소프트", "맥스", "가", " ", "뭐", "야", "?"]

↓ 사전에서 번호 찾기

[18204, 7731, 211, 9930, 402, 30]

(번호는 지어낸 값이에요.)

여기서 꼭 기억할 것: 사전 크기 = 출력층 칸 수

토큰 사전에는 보통 수만~십수만 개 조각이 들어 있어요. 이 개수가 나중에 4단계 **출력층 칸 수**가 돼요.

MNIST는 답이 "숫자 0~9" 중 하나라 출력이 10칸이었죠. 언어 모델은 답이 "사전에 있는 조각 중 하나"라서 출력이 사전 크기만큼이에요.

2단계 임베딩: 번호를 숫자 묶음으로

토큰 번호는 이름표일 뿐 의미가 없어요. 그래서 번호마다 **숫자 수천 개짜리 한 줄**을 짝지어 주는데, 이것을 **임베딩**이라고 해요.

왜 번호 그대로는 안 되나

"소프트"가 18204번, "맥스"가 7731번이라고 해볼게요. 이 번호를 그대로 계산에 넣으면 "18204가 7731보다 크다"는 뜻없는 크기 관계가 생겨요.

원-핫을 배울 때 "정답 7을 숫자 그대로 두면 7이 1보다 8에 가까운 것처럼 보인다"고 했죠. 같은 문제예요.

번호마다 숫자 한 줄을 꺼낸다

모델 안에는 큰 표(임베딩 표)가 있어요. 토큰 사전 크기만큼 줄이 있고, 한 줄에 숫자가 수천 개예요. 토큰 번호는 **이 표의 몇 번째 줄을 꺼낼지**를 가리켜요.

임베딩 표 (사전 크기 × 숫자 수천 개)

...

7731번 줄 ("맥스") → [0.71, 0.02, -0.41, ..., 0.90]

...

18204번 줄 ("소프트") → [0.12, -0.80, 0.33, ..., 0.05]

...

그래서 질문 토큰 6개는 이렇게 바뀌어요.

[18204, 7731, 211, 9930, 402, 30]

↓ 각 번호의 줄을 꺼냄

"소프트" → [0.12, -0.80, 0.33, ...]

"맥스" → [0.71, 0.02, -0.41, ...]

"가" → [-0.05, 0.44, 0.10, ...]

" 뭐" → [0.30, -0.12, 0.85, ...]

"야" → [-0.62, 0.09, -0.27, ...]

"?" → [0.18, 0.55, -0.03, ...]

= (6, 수천) 모양의 표

MNIST와 비교하면

- MNIST: 이미지 한 장 = 784개짜리 한 줄
- 언어 모델: 토큰 한 개 = 수천 개짜리 한 줄

질문은 토큰이 6개라서 줄이 6개 쌓인 표가 돼요. 배치 처리에서 이미지 100장을 쌓은 (100, 784) 표와 모양이 비슷하죠. 다만 여기서는 각 줄이 서로 다른 문장이 아니라, **한 문장 안의 단어 조각들**이에요.

이 숫자들은 누가 정했나

가중치와 똑같아요. 처음에는 작은 랜덤 값이고, **학습하면서 맞춰져요**. 학습이 끝나면 뜻이 비슷한 조각 ("날씨", "기후")은 비슷한 숫자 묶음을 갖게 돼요. 같은 문맥에서 자주 쓰이는 조각끼리 비슷해져야 다음 단어를 잘 맞힐 수 있기 때문이에요.

오해하기 쉬운 점 하나: 수천 개 숫자 하나하나에 "이건 성별, 이건 크기" 같은 이름이 붙어 있지는 않아요. 은닉층 뉴런처럼, 학습이 알아서 쓸모 있게 맞춘 숫자들이에요.

3단계 트랜스포머 ① 어텐션: 문맥을 섞는 장치

어텐션은 각 단어 조각이 **다른 조각들 중 누구에게 얼마나 집중할지를 소프트맥스로 정하고, 그 비율대로 정보를 섞는 장치**예요. 트랜스포머를 트랜스포머로 만드는 핵심 부품이에요.

왜 필요한가: 같은 단어도 문맥에 따라 뜻이 다르다

2단계의 임베딩은 토큰마다 **항상 같은 숫자 묶음**이에요. 그런데 말은 문맥을 봐야 뜻이 정해져요.

- "배가 고파서 밥을 먹었다" → 몸의 배
- "배를 타고 섬에 갔다" → 타는 배
- "나는 사과를 샀다. 그것은 아주 달았다" → "그것"은 사과

어텐션은 각 조각의 숫자 묶음에 **주변 조각의 정보를 섞어 넣어서**, "이 문장 안에서의 뜻"을 담은 숫자 묶음으로 바꿔줘요.

비유: 회의실에서 누구 말을 들을까

문장의 조각들이 한 회의실에 앉아 있다고 생각해 보세요. "그것은"이 자기 뜻을 알고 싶어서 다른 조각들을 둘러봐요.

1. 각자에게 "너 나랑 얼마나 관련 있어?" 점수를 매기고,
2. 그 점수를 **소프트맥스로 비율**(합 1)로 바꾸고,
3. 그 비율만큼 **각자의 말을 섞어서** 자기 것으로 만들어요.

이 세 단계가 어텐션의 전부예요. 숫자로 따라가 볼게요.

① 관련도 점수 매기기

점수는 두 숫자 묶음을 **같은 자리끼리 곱해서 더해요**. MNIST에서 픽셀과 가중치를 곱해 더한 것과 같은 계산이에요. 두 묶음이 비슷한 방향일수록 값이 커져요. "7번 뉴런 가중치가 7 모양과 겹칠수록 점수가 커진다"와 같은 원리예요.

"그것은"이 매긴 관련도 점수 (예시 값)

나는	1.0	
사과를	3.0	← 가장 관련 있음
샀다	1.5	
그것은	0.5	(자기 자신도 포함)

② 소프트맥스로 비율 정하기

3.5절에서 배운 그 소프트맥스 함수 그대로예요.

```
softmax([1.0, 3.0, 1.5, 0.5])
= [0.09, 0.69, 0.16, 0.06]    ← 합계 1
   나는 사과를 샀다 그것은
```

"그것은"은 "**사과를**"에 **69%** 집중하기로 해요.

③ 비율대로 섞기

설명을 위해 숫자 묶음을 2개짜리로 줄여볼게요.

나는 [1.0, 0.0]
 사과를 [0.0, 1.0]
 샀다 [0.5, 0.5]
 그것은 [0.2, 0.2]

섞은 결과 = $0.09 \times [1.0, 0.0] + 0.69 \times [0.0, 1.0] + 0.16 \times [0.5, 0.5] + 0.06 \times [0.2, 0.2]$
 $\approx [0.18, 0.78]$

섞인 결과 [0.18, 0.78]이 "사과를" [0.0, 1.0]에 가까워졌죠. 이제 "그것은"의 숫자 묶음 안에 **사과의 의미**가 녹아들었어요. 이 과정을 모든 조각이 동시에 해요.

소프트맥스의 두 가지 역할 비교

쓰이는 곳	역할	결과로 하는 일
출력층 (MNIST, 4단계)	후보들의 확률	하나를 고름
어텐션 (지금)	다른 조각에 집중할 비율	비율대로 섞음

같은 함수인데, "합이 1인 비율로 나눠주는" 성질을 다른 용도로 쓴 거예요.

실제로는 조금 더 있어요 (가볍게만)

- **가중치를 거친다:** 점수를 매기기 전에, 숫자 묶음에 학습된 가중치를 곱해서 "물어보는 용", "대답하는 용", "내용 전달용" 세 가지 버전을 만들어요. (전문 용어로 Query, Key, Value)
- **앞만 볼 수 있다:** 다음 토큰을 맞히는 모델이라, 각 조각은 자기보다 **앞에 있는 조각들**에만 집중할 수 있어요. 정답을 미리 보면 안 되니까요.
- **여러 개를 동시에:** 한 층 안에 어텐션이 여러 개 동시에 돌아요. 하나는 "그것이 무엇인지", 하나는 "문장의 시제", 하나는 "말투"를 보는 식으로 관점을 나눠 보게 돼요. (이름 붙인 관점은 이해를 돕기 위한 비유예요.)

3단계 트랜스포머 ② 블록을 여러 층 쌓기

트랜스포머는 "어텐션 + 곱하고 더하기" 한 세트(블록)를 수십 번 쌓은 것이예요. MNIST가 은닉층 2개를 쌓았다면, 언어 모델은 이 블록을 수십 개 쌓아요.

블록 하나 = "듣기" + "혼자 생각하기"

블록 하나는 두 부분으로 돼 있어요.

1. **어텐션 (듣기):** 앞 섹션에서 본 것. 각 조각이 다른 조각들의 정보를 비율대로 섞어와요. **조각끼리 정보가 오가는 유일한 곳**이예요.

2. 피드포워드 (혼자 생각하기): 각 조각이 자기 숫자 묶음만 가지고 곱하고 더하고 활성화 함수를 거쳐요. 섞어온 정보를 소화하는 단계예요.

회의에 비유하면, 다른 사람 말을 듣고(어텐션) 자기 자리에서 고개를 끄덕이며 정리하는(피드포워드) 한 라운드예요.

피드포워드는 사실 MNIST 신경망이에요

피드포워드는 3.6절 `predict()` 와 거의 같은 구조예요.

MNIST `predict()`:

`a1 = np.dot(x, W1) + b1`
(수) (수천)

`z1 = sigmoid(a1)`

`a2 = np.dot(z1, W2) + b2`

피드포워드 (조각 하나마다):

`a = np.dot(x, W1) + b1` (수천 → 더 큰

`z = 활성화함수(a)`

`y = np.dot(z, W2) + b2` (다시 수천으로)

활성화 함수는 시그모이드 대신 ReLU 비슷한 함수를 주로 써요(6장에서 나와요). 나머지는 우리가 아는 "곱하고 더하고 활성화"예요.

모양이 안 변해서 계속 쌓을 수 있다

블록에 (6, 수천) 표를 넣으면 **똑같이 (6, 수천) 표가 나와요**. 숫자 값만 더 풍부해져요. 그래서 나온 걸 다음 블록에 또 넣을 수 있어요.

(6, 수천) → [블록 1] → (6, 수천) → [블록 2] → (6, 수천) → ... → [블록 N] → (6, 수천)

블록마다 가중치는 따로예요. 블록이 수십 개라 가중치도 수십억~수천억 개까지 늘어나요. (MNIST 신경망은 약 4만 개였죠.)

층을 쌓을수록 무엇이 좋아지나

한 번 어텐션으로는 "그것 = 사과" 정도를 연결해요. 층을 거듭할수록 이미 문맥이 섞인 숫자 묶음끼리 다시 섞이니깐, 더 넓고 추상적인 관계를 담을 수 있어요.

예를 들면 "소프트맥스가 뭐야?"에서, 아래쪽 층에서는 "소프트" + "맥스"가 한 단어라는 것을, 위쪽 층에서는 "이건 개념을 설명해 달라는 질문이다" 같은 것을 담는 식이에요. MNIST 은닉층이 "가로선 → 가로선+대각선 조합"으로 올라간 것과 비슷한 비유예요. (층마다 이렇게 깔끔하게 나뉘지는 않지만, 감을 잡기엔 좋아요.)

두 가지 작은 장치

- **순서 정보 더하기:** 어텐션은 조각들을 순서 없이 모아서 보기 때문에, 그대로는 "개가 사람을 물었다"와 "사람이 개를 물었다"를 구분하기 어려워요. 그래서 각 조각에 **"나는 몇 번째 자리"**라는 정보를 숫자로 더해줘요.

- **원래 값에 더하기:** 블록은 숫자 묶음을 갈아어지 않고, 알아낸 것을 **원래 값에 더해요**. 층을 수십 개 쌓아도 처음 정보가 사라지지 않고, 학습도 안정적으로 돼요.

마지막 층을 지나면

수십 층을 지나면 **마지막 조각의 숫자 묶음**에는 그 앞의 내용 전체가 녹아들어 있어요. 어텐션으로 앞의 모든 조각을 보고 섞었으니까요. 다음 단계는 이 마지막 숫자 묶음 하나로 "다음 토큰"을 맞혀요.

4단계 출력층과 소프트맥스: 다음 토큰 확률

마지막 조각의 숫자 묶음을 출력층에 넣으면, ****사전에 있는 모든 토큰에 대해 "이게 다음에 올 점수"*****가 나와요. 이걸 소프트맥스로 확률로 바꿔요. 3.6절 `predict()` 의 마지막 두 줄과 똑같아요.

MNIST 출력층과 나란히 보기

```
MNIST:      a3 = np.dot(z2, W3) + b3      (100,) · (100, 10)      → (10,)
점수 10개

            y  = softmax(a3)                                → 확률 10개

언어 모델: a = np.dot(마지막조각, W출력)  (수천,) · (수천, 수만) → (수만,)   점수 수
만 개

            y  = softmax(a)                                → 확률 수만
개
```

계산은 같고 칸 수만 10개에서 수만 개로 늘었어요. 가중치 $W_{\text{출력}}$ 의 열 하나하나가 토큰 하나하나의 **기준** 표예요. "7번 뉴런이 7 모양에 높은 점수를 준다"처럼, "소프트" 토큰 열은 "여기서는 소프트가 올 차례다"는 문맥에 높은 점수를 줘요.

결과는 이런 모양

"소프트맥스가 뭐야?" 다음에 답변을 시작할 때라면 이런 식이에요. (확률은 예시 값이에요.)

토큰	확률
"소프트"	0.62
"좋은"	0.11
"네"	0.08
"Soft"	0.05
...	
"바나나"	0.00001
...	(수만 개 합계 = 1)

여기서 "예측"의 진짜 의미

언어 모델은 정답을 "알고" 말하는 게 아니에요. **학습한 수많은 글에서, 이런 문맥 다음에 어떤 조각이 왔는지를 바탕으로 확률을 매기는 거예요.**

그런데 수많은 글에서 "소프트맥스가 뭐야?"라는 질문 뒤에는 보통 소프트맥스의 설명이 왔겠죠. 그래서 다음 조각을 잘 맞히려면, 결국 **소프트맥스가 무엇인지를 모델이 숫자 안에 담고 있어야** 해요. "다음 조각 맞추기"만 시켰는데 지식과 문법이 따라서 쌓이는 이유가 이거예요.

반대로 주의할 점도 여기서 나와요. 모델은 "그럴듯한 다음 조각"을 고르는 것이라, 모르는 내용도 **그럴듯하게 지어낼 수 있어요**. MNIST가 고양이 사진을 받아도 숫자 중 하나를 골라버리는 것과 비슷해요. 흔히 "환각"이라고 부르는 현상이에요.

5단계 고르기: argmax, 추첨, temperature

확률이 나오면 **토큰 하나를 골라요**. MNIST는 무조건 1등(argmax)이었지만, 언어 모델은 고르는 방법이 여러 가지예요. 이해하기 쉽게 후보가 4개뿐이고 점수가 아래와 같다고 단순화할게요.

후보:	"소프트"	"좋은"	"네"	"바나나"	
점수:	3.0	2.5	2.0	0.5	
확률:	0.49	0.29	0.18	0.04	← softmax 결과

① argmax: 무조건 1등

`np.argmax` 로 가장 큰 것을 골라요. 항상 "소프트"예요.

- 장점: 같은 질문이면 항상 같은 답. 예측할 수 있고 안정적이에요.
- 단점: 문장이 딱딱하고, 같은 표현을 빙빙 돌며 반복하는 경우가 있어요.

② 추첨(샘플링): 확률대로 뽑기

확률을 **제비뽑기 확률**로 써요. 100번 뽑으면 대략 49번은 "소프트", 29번은 "좋은", 18번은 "네"가 나오는 식이에요.

"소프트"가 뽑히면 → "소프트맥스는 점수를 ..."
 "좋은"이 뽑히면 → "좋은 질문이에요. 소프트맥스는 ..."
 "네"가 뽑히면 → "네, 소프트맥스는 ..."

같은 질문에 ChatGPT나 Claude의 답이 **매번 조금씩 다른 이유**가 이거예요. 첫 조각이 다르게 뽑히면 그 뒤의 문장 전체가 달라져요.

③ temperature: 추첨을 얼마나 모험적으로 할지

소프트맥스에 넣기 **전에** 점수를 temperature(T)로 나눕니다. 점수 [3.0, 2.5, 2.0, 0.5]로 실제 계산한 값이에요.

T	점수 ÷ T	소프트	좋은	네	바나나	느낌
0.5	[6.0, 5.0, 4.0, 1.0]	0.66	0.24	0.09	0.00	1등에 물림, 안정적
1	[3.0, 2.5, 2.0, 0.5]	0.49	0.29	0.18	0.04	기본
2	[1.5, 1.25, 1.0, 0.25]	0.37	0.29	0.23	0.11	골고루 퍼짐, 다양함

왜 이렇게 되냐면, 소프트맥스는 **점수의 차이**에 민감하기 때문이에요.

- **T를 낮추면** (0.5) 점수가 2배로 커져서 차이도 2배가 돼요. 1등이 훨씬 유리해져요. T가 0에 가까워지면 사실상 argmax와 같아져요.
- **T를 높이면** (2) 점수가 절반으로 줄어서 차이도 줄어요. 등수끼리 비슷해져서 아래 후보도 자주 뽑혀요. 너무 높이면 "바나나" 같은 엉뚱한 단어도 나와요.

어떤 일에 어떤 값을 쓰냐면,

- 낮게: 코드 작성, 계산, 사실 질문처럼 **정답이 뚜렷한 일**
- 높게: 아이디어 내기, 이야기 쓰기처럼 **다양성이 중요한 일**

AI 모델을 API로 쓸 때 개발자가 직접 넣는 설정값이에요.

④ top-k / top-p: 꼬리 잘라내기

실제 후보는 4개가 아니라 수만 개예요. 하나 하나는 0.00001처럼 작아도, 수만 개가 모이면 "엉뚱한 것 중 하나가 뽑힐 확률"은 무시할 수 없어요. 그래서 추천 전에 꼬리를 잘라요.

- **top-k:** 확률 상위 k개만 남기고 나머지는 버려요. k = 3이면 "소프트", "좋은", "네"만 남아요.
- **top-p:** 확률 높은 순으로 더해서 합이 p가 될 때까지만 남겨요. p = 0.9이면 0.49 → 0.78 → 0.96에서 0.9를 넘으니 역시 세 개만 남아요.

남은 후보끼리 다시 합이 1이 되게 맞춘 다음 추천해요. (세 개의 합 0.96으로 나눠서 0.51, 0.30, 0.19.) 둘 다 소프트맥스 결과를 크기순으로 정렬해서 자르는 거예요.

6단계 질문 하나에 답하는 과정, 처음부터 끝까지

답변은 **미리 통째로 준비되어 있는 게 아니라, 1~5단계를 한 바퀴 돌 때마다 토큰 하나씩 만들어져요.** 만든 토큰을 끝에 붙이고, 그 길어진 글을 다시 처음부터 넣어서 다음 토큰을 만들고, 이걸 반복해요.

질문은 이런 모양으로 모델에 들어간다

사용자가 질문을 보내면, 앱이 대화를 **하나의 긴 글**로 만들어요. 누가 말했는지는 특별한 표시 토큰으로 붙여요.

[사용자] 소프트맥스가 뭐야?

[AI]

모델이 받는 건 이 글이 전부예요. 모델이 하는 일도 딱 하나, *******이 글의 다음 토큰은?"***이에요.

그런데 글이 [AI] 로 끝났죠. 학습한 대화 데이터에서 [AI] 다음에는 항상 **질문에 대한 답변**이 왔어요. 그래서 "다음 토큰 맞추기"를 하면 자연스럽게 **답변의 첫 토큰**이 나와요. 사용자의 다음 질문을 예상하는 게 아니라, **답변 자체를 이어 쓰는** 거예요.

토큰 하나씩 만들어지는 모습

각 회차는 1~5단계를 통째로 한 번 도는 거예요. (토큰 자르는 위치는 예시예요.)

회차	모델에 들어가는 글	고른 토큰
1	[사용자] 소프트맥스가 뭐야? [AI]	소프트
2	[사용자] 소프트맥스가 뭐야? [AI] 소프트	맥스는
3	[사용자] 소프트맥스가 뭐야? [AI] 소프트맥스는	점수를
4	[사용자] 소프트맥스가 뭐야? [AI] 소프트맥스는 점수를	확률로
5	[사용자] 소프트맥스가 뭐야? [AI] 소프트맥스는 점수를 확률로	바꿔주는
...	...	...
마지막	[사용자] ... [AI] 소프트맥스는 점수를 확률로 바꿔주는 함수예요.	[답변 끝]

[답변 끝] 도 사전에 있는 토큰 중 하나예요. 이게 뽑히면 멈추고, 앱이 [AI] 뒤의 내용을 답변으로 화면에 보여줘요.

한 회차 안에서 일어나는 일 (3회차 예시)

- 토큰화:** 지금까지의 글을 조각내고 번호로 바꿔요.
- 임베딩:** 번호마다 숫자 수천 개짜리 줄을 꺼내 표를 만들어요.
- 트랜스포머:** 수십 개 블록을 지나며 어텐션으로 문맥을 섞어요. 마지막 조각 "맥스는"의 숫자 묶음에 "질문은 소프트맥스의 뜻, 지금은 설명 문장의 주어를 막 끝냈다"는 정보가 담겨요.
- 출력층 + 소프트맥스:** 그 숫자 묶음으로 수만 개 토큰의 확률을 계산해요. "점수를", "입력값을", "신경망의" 같은 후보가 높아요.
- 고르기:** temperature를 적용해 추천하니 "점수를"이 뽑혔어요.
- 붙이기:** "점수를"을 글 끝에 붙이고 4회차로 넘어가요.

그래서 생기는 현상들

- **답변이 한 글자씩 흘러나오듯 보인다:** 실제로 토큰 하나씩 만들어지는 대로 화면에 보여주기 때문이에요.
- **답변이 길수록 오래 걸린다:** 토큰 하나마다 신경망 전체를 한 번씩 돌리니까요. 답변이 500토큰이면 500번 돌아요.
- **앞에서 한 말에 맞게 이어간다:** 매 회차 질문과 지금까지 쓴 답변을 전부 다시 보고 고르니까요.
- **앞에서 한 말을 못 물린다:** 한 번 뽑은 토큰은 고치지 않고 그대로 이어 써요. 초반에 엉뚱한 게 뽑히면 그 뒤도 그걸 따라 이어질 수 있어요.

(매 회차 "처음부터 다시 넣는다"고 했지만, 실제 서비스는 앞부분의 계산 결과를 저장해 두고 새 토큰 부분만 계산해서 시간을 아껴요. 결과는 같고 속도만 빨라져요.)

대화가 이어지면

두 번째 질문을 하면, 앱이 앞의 대화까지 전부 붙여서 넣어요.

```
[사용자] 소프트맥스가 뭐야?
[AI] 소프트맥스는 점수를 확률로 바꿔주는 함수예요. ...
[사용자] 그럼 argmax는?
[AI] < 여기서부터 다시 토큰 하나씩
```

AI가 앞 대화를 "기억"하는 것처럼 보이는 건, 사실 **매번 앞 대화를 통째로 다시 읽고 있기 때문이에요**. 신경망의 가중치는 대화 중에 바뀌지 않아요. 3장에서 학습된 가중치를 불러와 추론만 한 것과 같아요.

그래서 한 번에 넣을 수 있는 글의 길이에는 한도가 있고, 대화가 아주 길어지면 앱이 앞부분을 요약하거나 잘라내기도 해요.

학습: 정답은 "실제로 다음에 온 토큰"

언어 모델의 가중치도 MNIST처럼 **랜덤에서 시작해 정답에 가까워지도록 고치면서** 만들어져요. 다른 점은 정답 레이블을 사람이 붙이지 않고 **글 자체에서 나온다는** 거예요.

글 하나가 문제집이 된다

MNIST는 사람이 이미지마다 "이건 7"이라고 정답을 붙였죠. 언어 모델은 글 한 문장에서 문제와 정답이 저절로 만들어져요.

학습 문장: "소프트맥스는 점수를 확률로 바꿔준다"

문제		정답
"소프트"	→	"맥스는"
"소프트맥스는"	→	" 점수를"
"소프트맥스는 점수를"	→	" 확률로"
"소프트맥스는 점수를 확률로"	→	" 바꿔준다"

인터넷의 글, 책, 코드 같은 방대한 글을 이렇게 쓰면 문제가 수조 개 단위로 생겨요. 사람이 라벨링할 필요가 없어서 이렇게 큰 규모로 학습할 수 있는 거예요.

정답은 원-핫

3.6절에서 물어보셨던 원-핫이 여기서 쓰여요. 정답 " 점수를"은 사전 크기만한 배열에서 **그 토큰 자리만 1인 배열**이에요.

	...	" 점수를"	...	" 입력값을"	...	"바나나"	...
정답 (원-핫):	...	1	...	0	...	0	...
모델 출력:	...	0.30	...	0.12	...	0.0001	...

MNIST에서 정답 7이 [0,0,0,0,0,0,0,1,0,0] 이었던 것과 같아요. 칸이 10개에서 수만 개로 늘었을 뿐이에요.

고치는 방법 (4장 내용)

1. 문제를 넣어서 출력 확률을 얻어요. (추론, 지금까지 배운 것)
2. 정답 자리 확률이 얼마나 낮은지 **손실 함수**로 재요. 위 예시는 정답 자리가 0.30이라 아직 부족해요.
3. 정답 자리 확률이 **조금 더 커지는 방향으로** 모든 가중치(임베딩, 어텐션, 피드포워드, 출력층)를 조금씩 고쳐요.
4. 이걸 배치로 묶어서 엄청나게 많이 반복해요.

이 절차는 MNIST 학습과 같아요. 다른 건 규모로, 문제 수도 가중치 수도 수억 배 이상 커서, 많은 GPU로 오랫동안 학습해요.

대화를 잘하게 만드는 추가 학습

위의 학습만 한 모델은 **"글 이어 쓰기"**는 잘하지만 **질문에 잘 답하는 건 아니에요**. 예를 들어 "소프트맥스가 뭐야?" 뒤에 비슷한 질문을 여러 개 더 나열할 수도 있어요. 인터넷에는 질문 목록도 많으니까요.

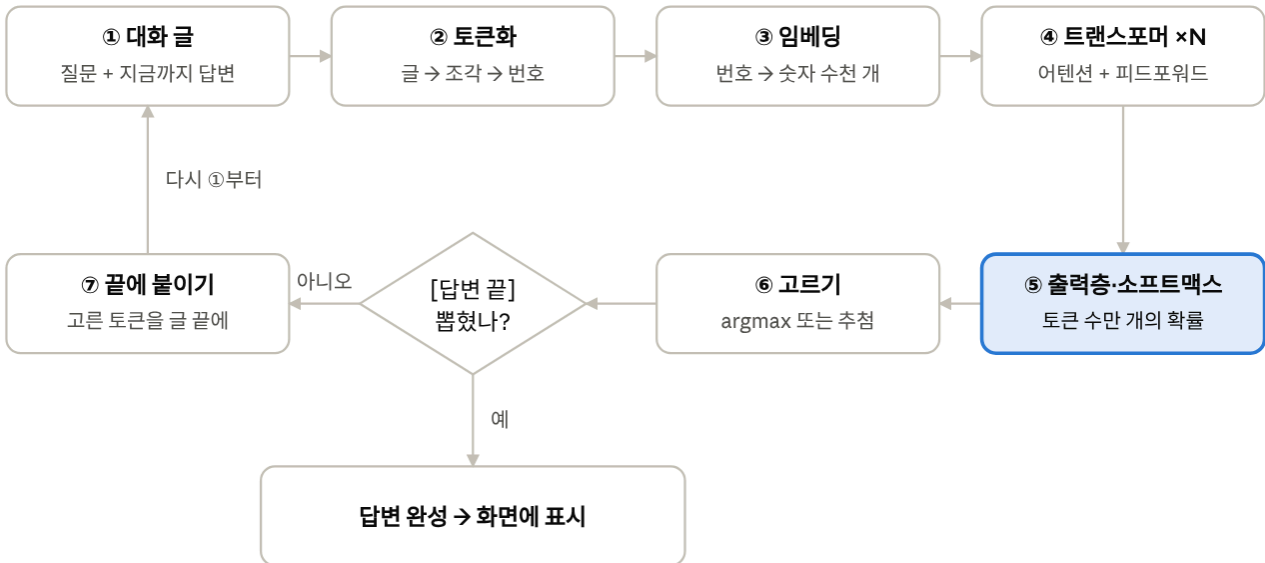
그래서 ChatGPT나 Claude 같은 대화형 모델은 한 번 더 학습해요. [사용자] 질문 [AI] 좋은 답변 형태의 예시로 학습하고, 사람이 더 도움이 된다고 판단한 답변 쪽으로 더 기울도록 고쳐요. 6단계에서 [AI] 뒤에 답변이 자연스럽게 나오는 건 이 추가 학습 덕분이에요. 하지만 추가 학습 후에도 답변을 만드는 방식 자체는 그대로 **다음 토큰 맞히기의 반복**이에요.

한눈에 정리

언어 모델은 3.6절의 추론(숫자로 바꾸기 → 곱하고 더하기 → 소프트맥스 → 고르기)을 토큰 하나마다 한 번씩, 답변이 끝날 때까지 반복하는 거예요.

답변은 다음 토큰 고르기를 반복해서 만들어진다

한 바퀴 = 토큰 하나. [답변 끝]이 났을 때까지 반복



언어 모델이 답변을 만드는 흐름 · 7단계 + 반복

①~⑥를 한 바퀴 돌 때마다 토큰이 하나 생기고, ⑦에서 글 끝에 붙여 다시 ①로 돌아가요.

기억할 것 세 가지:

- **소프트맥스는 두 곳에서 쓰여요.** 어텐션에서는 "누구에게 얼마나 집중할지" 비율을, 출력층에서는 "다음 토큰이 무엇일지" 확률을 만들어요.
- **답변은 미리 있는 게 아니에요.** [AI] 뒤에 올 토큰을 하나씩 고르며 이어 쓰는 거예요.
- **가중치는 대화 중에 바뀌지 않아요.** 학습은 미리 끝났고, 대화는 3장처럼 추론만 하는 거예요. 앞 대화를 아는 건 매번 다시 읽기 때문이에요.